FIT2109 Computer Science Workshop

Week 2: Shell Tools & Scripting

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

Bash is used everywhere — and it fails everywhere

Research finding

A study of real open-source Bash scripts found that common bugs involve quoting, word splitting, path and directory management, command options, return values, and error handling.

- These are not obscure edge cases. They appear in widely-used projects.
- The shell rewards careful habits and punishes careless assumptions.

Paper

Bash in the Wild: Language Usage, Code Smells, and Bugs — TOSEM 2022

A very short history of Bash

- Unix shells began as command-line interfaces for the operating system.
- The **Bourne shell** (`sh`) became a standard early Unix shell.
- **Bash** stands for **Bourne Again SHell**.
- Bash was created as a GNU replacement and extension of `sh`.
- It keeps backward compatibility with older shell ideas — which is why it is powerful but sometimes unintuitive.

Key point

Bash is both an interactive command interface and a scripting language. The same evaluation rules apply in both contexts.

Learning goals

By the end of this workshop, you should be able to:

- Explain quoting rules and predict how the shell evaluates a command.
- Use variables, `export`, and `$?` to control command-line behaviour.
- Construct `grep`, `find`, and text-processing pipelines using regular expressions.
- Write a basic shell script with arguments, conditionals, and loops.
- Apply `set -u` and `set -o pipefail` to improve script reliability.
- Use `ssh` and `scp` to access remote machines and transfer files.

Poll Everywhere · Word Cloud

pollev.com/fit2109

Type **one thing** the shell does that you could not do in a GUI.

Concept 1: The shell rewrites before it runs

Shell evaluation model

The shell processes a command line in stages, not all at once.

1. **Read** the input line.
2. **Parse** quoting and structure.
3. **Expand**: variables, globs, command substitution.
4. **Split** unquoted results on whitespace.
5. **Execute** the resulting command with its arguments.

Practical consequence

What you type is not always what the shell runs. Quoting controls what gets expanded or split.

Concept 2: Quoting changes what the shell sees

No quotes

Shell expands and word-splits.

`$name` → split on spaces
`*` → match filenames

Double quotes

Variables expand. Spaces preserved. Glob does not expand.

`"$name"` → one argument

Single quotes

Everything is literal. No expansion at all.

`'$name'` → the text `$name`

Professional habit

Always quote variables that may contain spaces or special characters: `"$file"`, `"$1"`.

Demo 1: quoting traps

Live on Terminal

Watch what the shell actually receives as arguments.

```
mkdir -p "Week 2 Materials"
touch "Week 2 Materials/errors log.txt"
file="Week 2 Materials/errors log.txt"

ls $file          # dangerous: word-splits into two arguments
ls "$file"        # safe: treated as one argument

echo *.txt        # glob expands to matching filenames
echo "*.txt"      # literal text

echo '$file'      # single quotes: no expansion
echo "$file"      # double quotes: variable expands
```

Which correctly handles a filename that contains spaces?

A `ls $file`

B `ls "$file"`

C `ls '$file'`

D `ls {$file}`

Activity 1: predict the shell's behaviour

Run this first if the variable is not already set

```
file="Week 2 Materials/errors log.txt"
```

In pairs: what does each command do?

1. `cd Week 2 Materials`
2. `cd "Week 2 Materials"`
3. `echo $file`
4. `echo "$file"`
5. `echo '*.txt'`

Poll Everywhere · Quick Text

pollev.com/fit2109

Type the command that lists **all current environment variables**.

Concept 3: Variables can stay local or travel

Shell variable

Visible only in the current shell.

```
course=FIT2109
echo "$course"
```

Environment variable

Inherited by child processes after `export`.

```
export course
bash -c 'echo $course'
```

Per-command environment

```
COURSE=FIT2109 env | grep ^COURSE
```

- `env` lists the current environment.
- A prefix variable is visible to that one command only.

Demo 2: variables, export, and child shells

Live on Terminal

Watch which shells can see the variable.

```
course=FIT2109
echo "$course"
bash -c 'echo "Child sees: $course"'   # empty before export

export course
bash -c 'echo "Child sees: $course"'   # now visible

COURSE=FIT2109 env | grep '^COURSE='   # per-command only
echo "$COURSE"                          # empty in parent shell
```

Student check

Why does the child shell see an empty value before `export`?

A child shell can see a variable only after the parent runs:

A `set var=value`

B `local var`

C `export var`

D `source var`

Concept 4: Exit status lets commands make decisions

Key idea

Every command returns an exit status. `0` means success; nonzero means failure. The shell can branch on this automatically.

- `$?`: the exit status of the last command.
- `cmd1 && cmd2`: run `cmd2` only if `cmd1` succeeded.
- `cmd1 || cmd2`: run `cmd2` only if `cmd1` failed.

Common pattern

```
cd "$dir" || exit 1
```

- If `cd` fails, the script exits immediately instead of running the next command in the wrong directory.

Demo 3: exit status in practice

Live on Terminal

Connect exit status to command-line decision making.

```
mkdir demo_dir && echo "created"
echo $?
mkdir demo_dir    # fails: already exists
echo $?

grep -q "ERROR" logs/app-2026.log && echo "Errors found"
grep -q "CRITICAL" logs/app-2026.log || echo "No CRITICAL lines"
cd /tmp/nonexistent || exit 1
```

Question

What is the exit status of `grep -q` when it finds a match? When it does not?

Concept 5: Globs and regexes are different languages

Shell glob

Expanded by the shell *before* the command runs. Matches filenames.

`*.txt` matches files ending in `.txt`

Regular expression

Interpreted by the tool, not the shell. Matches text inside files.

`.*\.txt$` matches the string `.txt` at end of a line

Common mistake

Confusing `*.log` (shell glob) with `.*\.log` (regex). Always quote regex patterns so the shell does not try to expand them: `grep -E 'pattern'`.

The glob `*.log` is expanded by:

A The shell, before the command runs

B The tool (`grep`, `find`, etc.)

C Both, independently

D The kernel

Concept 6: grep flags shape the question

- `-E`: extended regex (alternation, +, ?)
- `-o`: print only the matched text
- `-n`: show line numbers
- `-q`: quiet; exit status only — useful in scripts
- `-r`: search recursively through directories
- `-i`: case-insensitive match

Useful patterns

`grep -E 'WARN(ING)?'` matches `WARN` and `WARNING`

`grep -Eo 'ERROR [0-9]+'` extracts just the error code

Demo 4: grep on a log file

Live on Terminal — working in the demo workspace

Each flag changes what gets printed — and whether the exit status signals success.

```
grep '^ERROR' server.log
grep -E 'WARN(ING)?' server.log
grep -Eo 'ERROR [0-9]+' server.log
grep -Eo 'ERROR [0-9]+' server.log | sort | uniq -c | sort -nr
grep -rEn 'TODO|FIXME' src/
```

Question

Which of these commands produces output, and which only sets the exit status?

Poll Everywhere · Word Cloud

pollev.com/fit2109

Type **one** `grep` **flag** you will remember from today.

Concept 7: `find` searches by file metadata

Key idea

`find` locates files using filesystem properties — name, type, permissions — not by searching file contents.

- `-name '*.log'`: match filename pattern (quote the glob).
- `-type f` / `-type d`: files or directories only.
- `! -perm -u=x`: files not executable by owner.
- `-regex`: match the full path with a regex.
- `-exec cmd {} +`: run a command on each result.

Important

`find -regex` matches the *full path*, not just the filename. The pattern must account for the leading path.

Demo 5: `find` examples

Live on Terminal

```
find . -type f -name '*.py'
find . -type f -name '*.sh' ! -perm -u=x
find . -regextype posix-extended \
     -type f -regex '.*/[a-z]+-[0-9]{4}\.log'
find . -type f -name '*.py' -exec grep -nE 'TODO|FIXME' {} +
```

Question

Why does `find -regex` need the leading path in the pattern, not just the filename part?

Poll Everywhere · Multiple Choice

pollev.com/fit2109

`find . -regex '.*error.*'` — does it match `./logs/error.log`?

A Yes, always

B No — `-regex` matches the full path, not just the filename

C Only with `-type f`

D Only on Linux

Activity 2: design a pipeline

Dataset: `server.log`

```
ERROR 404 Page not found      (appears twice)
ERROR 500 Database unavailable
WARNING CPU temperature high
```

Tasks — write the commands

1. A pipeline that counts how many times each distinct error code appears, sorted highest first.
2. A `find` command that locates shell scripts not executable by their owner.

Available: `grep -Eo`, `sort`, `uniq -c`, `sort -nr`, `find`, `! -perm`

Concept 8: Scripts make workflows reusable

Key idea

A shell script is a text file the shell reads and evaluates, exactly as if you typed each command yourself.

- **Shebang** on line 1 names the interpreter: `#!/usr/bin/env bash`
  macOS users: this ensures Bash runs the script even if your interactive shell is Zsh.
- `$1`, `$2`, ...: positional arguments.
- `$#`: number of arguments.
- `${1:-default}`: use `default` if `$1` is unset or empty.
- `if/then/else/fi`: conditional.
- `for x in ...; do ...; done`: loop.
- `-f "$file"`: test whether a file exists.

Concept 9: Reliable scripts fail clearly

Three habits that prevent common failures

- `set -u`: fail immediately if an unset variable is used. Prevents silent empty-string bugs.
- `set -o pipefail`: fail if any command in a pipeline fails, not just the last one.
- Quote every variable: `"$1"`, `"$file"` — not `$1`, `$file`.

Safe script header

```
#!/usr/bin/env bash
set -u
set -o pipefail
```

Demo 6a: a script with a default argument

Live on Terminal

A minimal script that still handles missing input gracefully.

```
#!/usr/bin/env bash
name=${1:-world}
echo "Hello, $name"
```

```
chmod u+x scripts/greet.sh
./scripts/greet.sh
./scripts/greet.sh Dennis
```

Question

What does `${1:-world}` do when the script is called with no arguments?

Demo 6b: add safety to a file-check script

Live on Terminal — apply the three habits from Concept 9

```
#!/usr/bin/env bash
set -u
set -o pipefail
file="$1"
if [ -f "$file" ]; then
    wc -l < "$file"
else
    echo "File not found: $file" >&2
    exit 1
fi
```

Compare

What breaks if you remove the quotes around `"$file"`?

Activity 3: review a bad script

```
#!/usr/bin/env bash
file=$1
if [ -f $file ]; then
    grep ERROR $file | sort | uniq -c
else
    echo File not found: $file
fi
```

Discuss in pairs — find at least four problems

1. What breaks if the filename contains spaces?
2. Why should the error message go to standard error?
3. What exit status should the script return on failure?
4. How do `set -u` and `set -o pipefail` help?

Find **at least one problem** in this script fragment: `if [ -f $file ]; then grep ERROR $file; fi`

Rank these script safety habits from **most** to **least** important to you:

- `set -u`
- `set -o pipefail`
- Quoting every variable: `"$var"`
- Using `#!/usr/bin/env bash`

Concept 10: SSH uses keys to trust remote machines

Key idea

SSH encrypts the connection between two machines. Key-based authentication replaces repeated password prompts with a cryptographic proof.

Private key

Stays on your machine only. Never share it. Protect it like a password.

Public key

Copied to the remote server. Goes into `~/.ssh/authorized_keys`.

- The server grants access when your private key matches the stored public key.
- One key pair can authenticate to many servers.

Demo 7: ssh-keygen, ssh, and scp

```
# Generate a key pair (user@host = 127.0.0.1 in the applied class)
ssh-keygen -t ed25519 -C "your@email.com"
ls -la ~/.ssh

# Copy public key to the server
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@host

# Connect interactively, or run a single command
ssh user@host
ssh user@host 'pwd && uname -a'

# Transfer files
scp scripts/deploy.sh user@host:/tmp/
scp user@host:/tmp/result.log .
```

Activity 4: plan a remote workflow

Scenario

You have a script called `deploy.sh`. You need to:

1. Copy it to a remote Linux server.
2. Run it there.
3. Inspect the last 20 lines of a deployment log.
4. Copy the log file back to your machine.

In pairs

Write the sequence of `scp` and `ssh` commands. Why is key-based authentication especially useful in a scripted workflow?

What should feel different after today?

- You can explain: **why a command behaves differently with and without quotes**.
- You can use: `$?`, `&&`, `||` **to handle success and failure**.
- You can build: **a** `grep`/`find`/`sort`/`uniq` **pipeline to answer a question from text**.
- You can write: **a small script with arguments, conditionals, and safety flags**.
- You can connect: **to a remote machine with SSH and transfer files with** `scp`.

The bigger idea

Bash rewards precision. Quoting, exit status, and safety flags are not optional extras — they are the difference between a script that works and one that silently corrupts data.

Exit ticket

1. What is the difference between `echo $file` and `echo "$file"`?
2. What does exit status `0` mean, and how does `&&` use it?
3. What is the difference between a shell glob (`*.log`) and a regex (`.*\.log$`)?
4. What does `set -u` protect against?
5. What is the difference between a public key and a private key in SSH?
6. Which Week 2 topic do you want to practise more in the applied class?

A script with **no** `set -o pipefail` runs `grep ERROR missing.log | wc -l`. The file does not exist. What does `echo $?` print?

A 0

B 1

C 2

D Depends on the shell

Q&A and next steps

Questions?

Before the applied class

Practise the quoting and scripting tasks until you can explain not just what a command does, but why it is written that way.